面向对象分析与设计
Object Oriented Analysis and Design

——函数式编程
Functional Programming

邱明 博士
厦门大学信息学院
mingqiu@xmu.edu.cn
2025年秋季学期

- Neal Ford. Functional Thinking. O'Reilly Media, Inc. 978-1-449-36551-6, 2014



- 函数式设计(原则模式与实践英文版)/经典原版书库
- 作者:(美)罗伯特·C.马丁
- ISBN：9787111760627
- 出版年月：2024-10
- 出版社：机械工业出版社

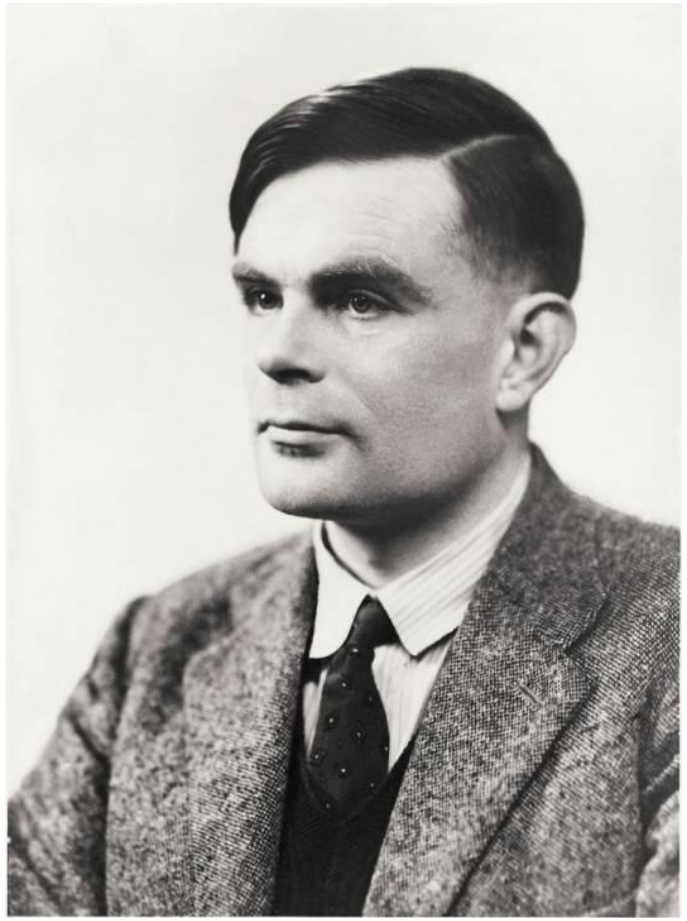


1. 函数式编程的起源

the Origin of Functional Programming

1.1 起源
origin

- 1936年艾伦.图灵（Alan Mathison Turing，1912/6/23-1954/6/7）在论文“On Computable Numbers, with an Application to the Entscheidungsproblem”发明了一种数字计算机，并证明了即使给定无限的时间和空间，计算机也无法计算某些数字。



1.1 起源
origin

- 1936年阿隆佐·邱奇（Alonzo Church, 1903—1995）提出无类型lambda演算，作为解决希尔伯特判定问题的形式系统。同时提出对判定问题的否定证明，指出不存在通用算法能判定所有lambda表达式的等价性。
- lambda演算与图灵机在计算能力上等价，二者可相互模拟。

1.1 起源
origin

- 来自 Robert Floyd 在 1978 年12月4日图灵奖的颁奖演说（The Paradigms of Programming）
- 是指程序的设计方法，即程序应该如何被构建和执行



1.2 主要的编程范式
Three Paradigms

- 结构化编程（Structured Programming）
- 面向对象编程（Object-Oriented Programming）
- 函数式编程（Functional Programming）

> OO makes code understandable by encapsulating moving parts. FP makes code understandable by minimizing moving parts.
>
> — Michael Feathers

Few Data Structures, Many Operations

2.函数式编程

Functional Programming

2.1 基本思想
Basic Ideas

- 用函数进行编程
- 函数是“一等公民”元素
  - 可以作为参数传值，也可以作为返回值
- 具有引用性透明的编程方式
- 基于lambda演算的编程风格

2.1 基本思想
Basic Ideas

- 结构化编程和面向对象编程中的函数只是一个将一系列顺序执行的指令包裹起来的过程
- 函数式编程是使用纯函数操作不可变值的编程方法
- 函数式编程中的函数与数学中的函数定义相同
  - 函数是将输入映射到输出的数学对象

2.2 纯函数
Pure Functions

- 函数是将输入映射到输出的数学对象

$$f(x,y) = y\sin(x)$$

$$g(x) = log(x)$$

$$g\big(f(x,y)\big) = \log(ysin(x))$$

$$h(x) = \cos(x)$$

$$g\big(h(x)\big) = \log(\cos(x))$$

2.2 纯函数
Pure Functions

- 所有的数学函数都是纯函数
  - 纯函数始终只返回一个值
    - 单一职责
  - 纯函数仅基于其参数计算返回值
  - 纯函数不会改变任何现有值
- 纯函数是无副作用的
- 纯函数是引用透明的
  - 始终可以用函数的返回值替换函数调用本身

2.2 纯函数
Pure Functions

- Java函数式接口
  - 在Java 8中引入的函数式编程定义
  - 只能有一个抽象方法
  - 可以有任意数量的默认方法
  - 可以有任意数量的静态方法
  - 从Object类继承的方法

```
@FunctionalInterface
public interface Comparator<T> {

    int compare(T o1, T o2);

    boolean equals(Object obj);

    default Comparator<T> reversed() {
        return Collections.reverseOrder(this);
    }
}
```

2.2 纯函数
Pure Functions

- Java lambda表达式
  - 当函数作为参数和返回值，lambda表达式是最好的武器。

```
List<String> keys = subRegions.stream().filter(o -> !o.getStatus().equals(Region.ABANDONED))
        .flatMap(subRegion -> {
            List sub = null;
            if (subRegion.allowTransitStatus(status)) {
                sub = subRegion.changeStatus(status, user);
                return (Stream<String>) sub.stream();
            }else{
                return new ArrayList<String>(0).stream();
            }
        }).distinct().collect(Collectors.toList());
```

2.3 不变性
Immutability

- 纯函数实现不可变性的方式
  - 通过副本来对抗可变性
    - 纯函数不修改旧的值，只产生新的值
  - 通过使用不可变值来对抗可变性
    - 纯函数仅接受不可变值（final），并返回不可变值

2.3 不变性
Immutability

- 指针的恶梦
  - 可以用指针传值，意味着函数可以修改从外部传入的变量。
  - 用指针传值的变量是在多个函数间共享的，这些共享变量的值会影响函数的执行结果。

2.3 不变性
Immutability

- 闭包（Closure）
  - 如果函数或代码块引用了的外部变量，则函数和其引用的外部变量一起构成闭包，即使外部上下文销毁，闭包仍能访问和操作这些变量。
  - Java 中没有原生闭包，通过匿名内部类和Lambda 表达式可实现 “闭包特性”

2.3 不变性
Immutability

```
public static Collection<Collection<ProductItem>> gotProductItems(Collection<ProductItem> productItems, Collection<Collection<Item>> items) {
    return items.stream().map(pack -> pack.stream().map(item ->
            productItems.stream().
                    filter(prodItem -> prodItem.getOrderItemId() == item.getId() && prodItem.getProductId() == item.getProductId())
                    .limit(1)
                    .map(productItem -> {
                        try {
                            return productItem.cloneWithQuantity(item.getQuantity());
                        } catch (CloneNotSupportedException e) {
                            throw new RuntimeException(e);
                        }
                    })
                    .collect(Collectors.toList()).get(0)
    ).collect(Collectors.toList())).collect(Collectors.toUnmodifiableList());
}
```

- 捕获的局部变量必须是 effectively final：
  - 要么显式加 final 关键字；
  - 要么未加 final 但从未被重新赋值（Java 8+ 支持，编译器自动判定）

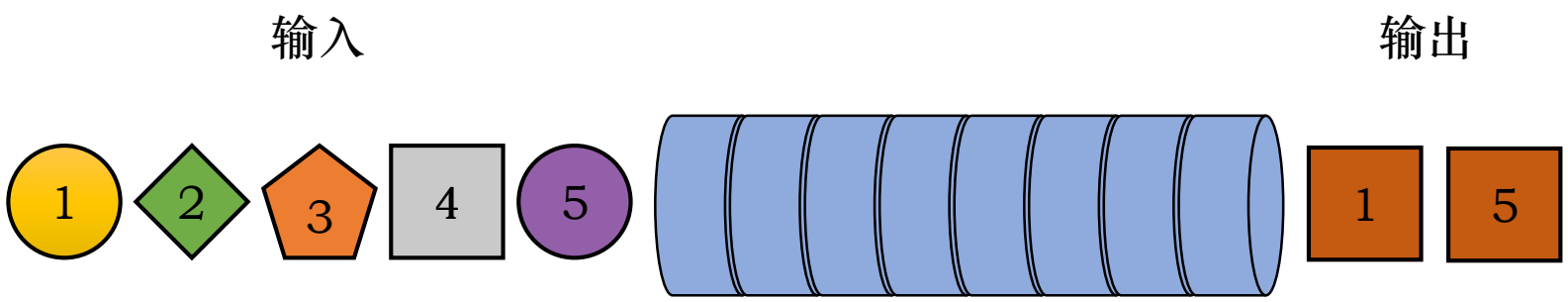
2.4 高阶函数
High Order Operation

- 能接受函数作为参数或能够把函数作为返回值的函数。
  - Java Stream的大多数方法都返回一个新的Stream对象
  - 每个方法都不改变任何内容，返回一个新的不可变对象

```
return this.regionDao.retrieveSubRegionsById(id, page, pageSize).stream()
                .filter(region -> region.getStatus().equals(Region.VALID))
                .map(o -> IdNameTypeVo.builder().id(o.getId()).name(o.getName()).build())
                .collect(Collectors.toList());
```

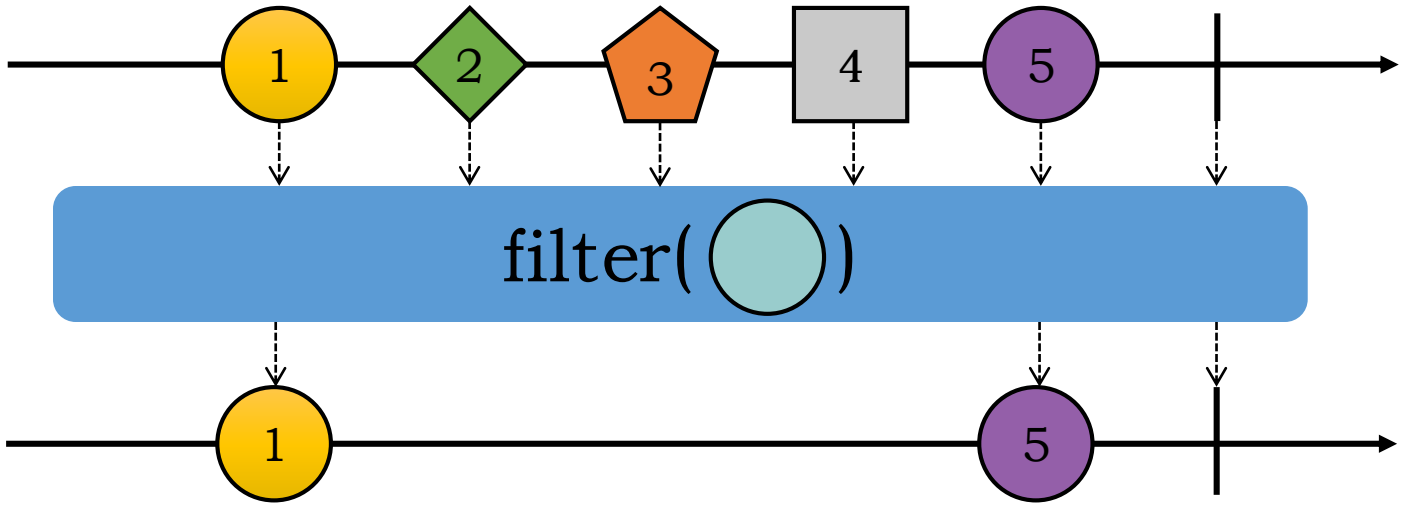
2.4 高阶函数
High Order Operation

- Java中Stream
  - 作为输入和输出之间的管道，不存储值
  - 不修改输入，产生新的输出
  - 惰性计算



2.4 高阶函数
High Order Operation

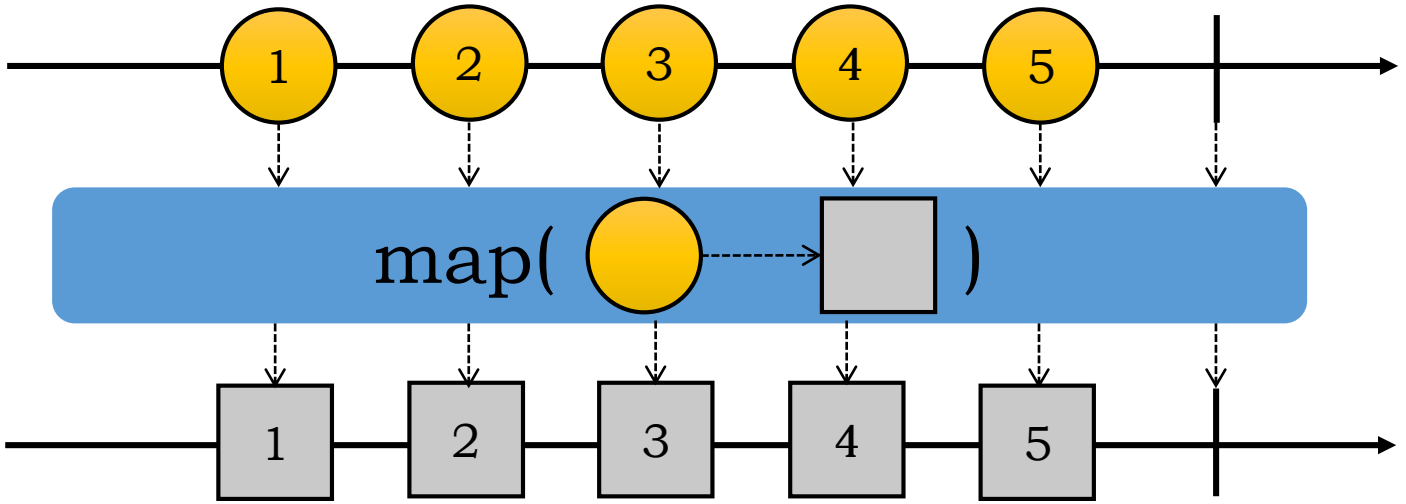
- 过滤（filter）




```
onsaleVo.getActList().stream().filter(activity -> activity.getId() == item.getActId()).count()  > 0
```

2.4 高阶函数
High Order Operation

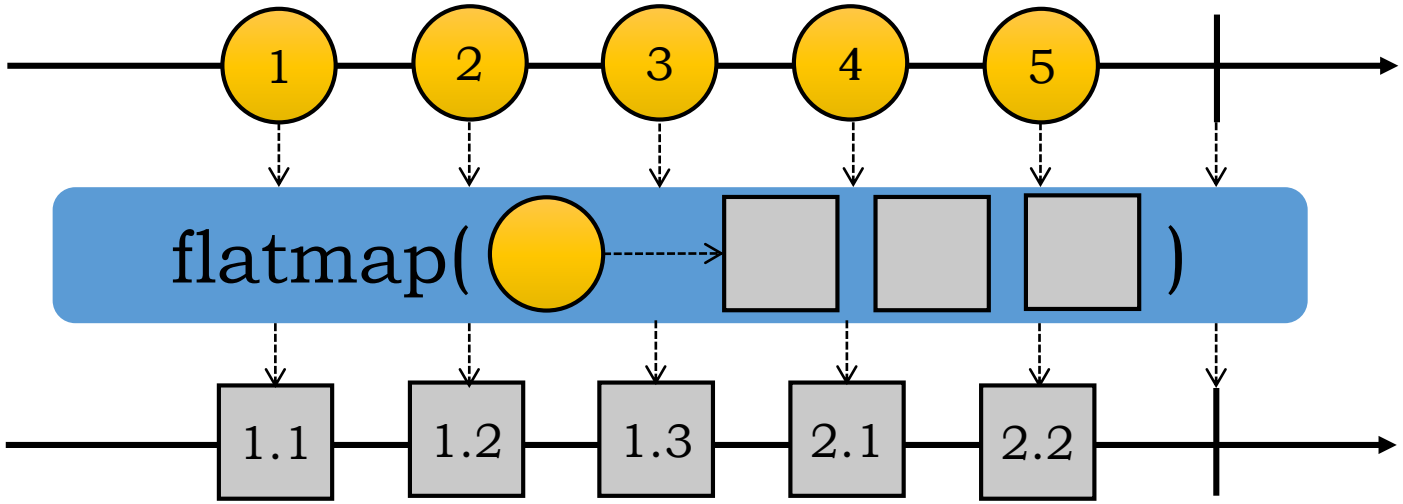
- 转换（map）




```
List<SimpleProductVo> vos = products.stream().map(obj -> new SimpleProductVo(obj)).collect(Collectors.toList());
```

2.4 高阶函数
High Order Operation

- 平铺转换（flatmap）

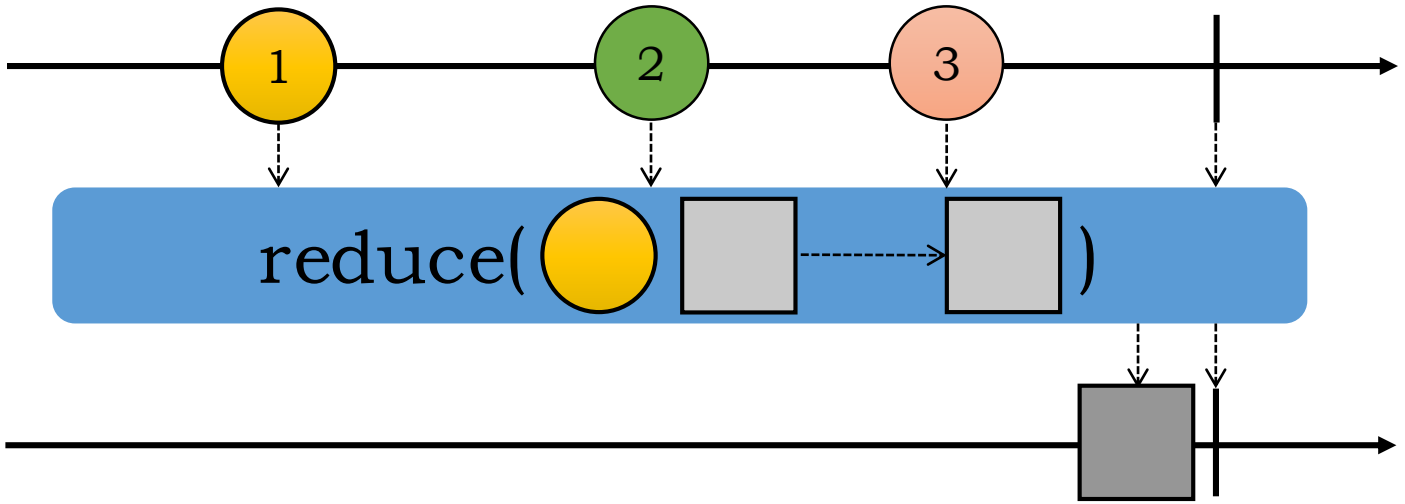


2.4 高阶函数
High Order Operation

```
List<String> keys = subRegions.stream().filter(o -> !o.getStatus().equals(Region.ABANDONED))
        .flatMap(subRegion -> {
            List sub = null;
            if (subRegion.allowTransitStatus(status)) {
                sub = subRegion.changeStatus(status, user);
                return (Stream<String>) sub.stream();
            }else{
                return new ArrayList<String>(0).stream();
            }
        }).distinct().collect(Collectors.toList());
```

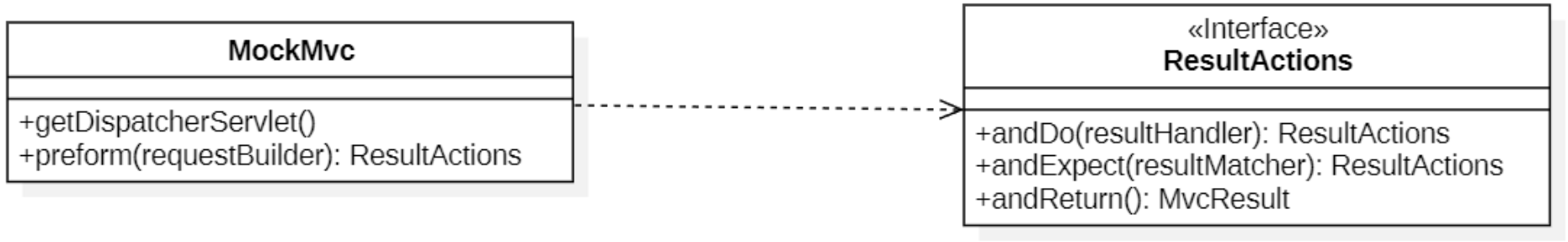
2.4 高阶函数
High Order Operation

- 递归（reduce）




```
this.getRefundTransList().stream()
    .filter(trans -> RefundTrans.FAIL != trans.getStatus())
    .map(RefundTrans::getAmount)
    .reduce((x,y)->x + y)
    .orElse(0L);
```

2.4 高阶函数
High Order Operation




```
this.mockMvc.perform(MockMvcRequestBuilders.get(LEDGER, 1)
                .header("authorization", adminToken)
                .contentType(MediaType.APPLICATION_JSON_VALUE)
                .param("beginTime", "2022-11-06T12:00:00")
                .param("endTime", "2022-11-09T12:00:00")
                .param("type", "0")
                .param("channelId", "501"))
        .andExpect(MockMvcResultMatchers.status().isOk())
        .andExpect(MockMvcResultMatchers.content().contentType("application/json;charset=UTF-8"))
        .andExpect(MockMvcResultMatchers.jsonPath("$.errno", is(ReturnNo.OK.getErrNo())))
        .andExpect(MockMvcResultMatchers.jsonPath("$.data.list[?(@.id == '%d')].transNo", 501).value("1111"));
```

2.5 柯里化
Curry

- 函数的返回值也可以是一个函数
- 将接受多个参数的函数转换成为一系列接受单个参数的函数。
- 柯里化后的函数不会立刻执行，而是返回一个新的函数，等到后面调用的时候在执行。
- 分解函数——允许将函数部分应用于某些参数，以创建新的函数

2.5 柯里化
Curry

- 哈斯凯尔·柯里，Haskell Brooks Curry，1900/9/12－1982/9/1，美国马萨诸塞州米里镇人，数理逻辑学家，专长于组合子逻辑理论。

